

Transcription automatique audio guitare → MIDI

Soutenance pour le titre professionnel *Concepteur Développeur en Science des Données*

Damien DESSAUX

M2i

3 septembre 2026

1. Introduction
2. BC01 - Construction et alimentation d'une infrastructure de gestion de données
3. BC02 - Analyse exploratoire, descriptive et inférentielle de données
4. BC03 - Analyse prédictive de données structurées par l'intelligence artificielle
5. BC04 - Analyse prédictive de données non-structurées par l'intelligence artificielle
6. BC05 - Industrialisation d'un algorithme d'apprentissage automatique et automatisation des processus de décision
7. BC06 - Direction de projets de gestion de données
8. Conclusion

Introduction du projet

Contexte métier

La retranscription manuelle d'un audio de guitare vers un fichier MIDI est une tâche **chronophage**, nécessitant une **expertise musicale**. Une minute d'audio peut nécessiter **15 à 30 minutes de retranscription** selon la complexité du morceau.

Projet GuitarFlow

GuitarFlow souhaite développer un **Proof Of Concept** capable de transcrire automatiquement un audio de guitare en un fichier MIDI exploitable par un logiciel de MAO (Musique Assistée par Ordinateur). Le projet vise à réduire le temps de transcription, accélérer la création de supports pédagogiques ou encore faciliter les workflows de composition musicale.

Objectif du POC

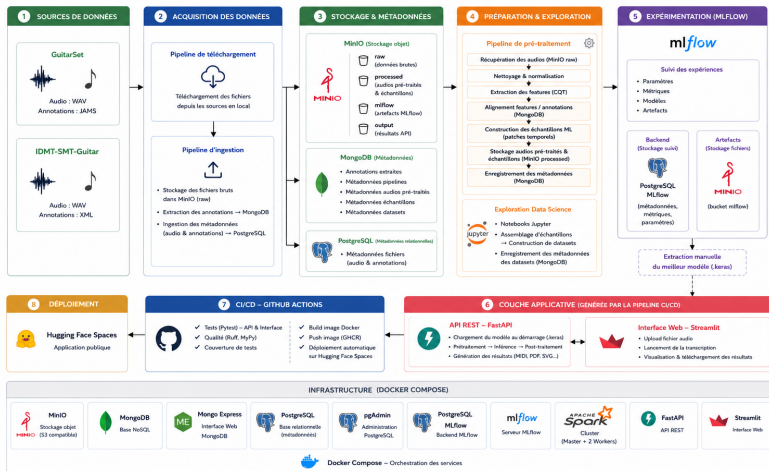
Valider la faisabilité technique d'une solution de Machine Learning de transformation audio vers MIDI, avant une éventuelle phase d'industrialisation.

BC01 - Construction et alimentation d'une infrastructure de gestion de données

Infrastructure conceptualisée

ARCHITECTURE GLOBALE DE LA PLATEFORME

Guitarflow – Projet de transcription musicale



Composants principaux

- Data Lake : MinIO
- Bases de données : MongoDB & PostgreSQL
- Traitement : Python / Spark
- Expérimentation : MLflow
- Infrastructure : Docker-Compose

Choix d'architecture

Séparation des données, métadonnées et traitements afin de garantir **traçabilité**, **évolutivité** et **reproductibilité**.

Projection de l'architecture vers AWS

Objectif : Conserver les principes de l'architecture du POC tout en préparant une première mise en production cloud.

Dimensionnement cible

- 1 000 audios/jour
- Durée moyenne : 1 min
- Taille moyenne : 10 Mo/audio
- Volume entrant : 300 Go/mois
- Inférence : 4 vCPU / 16 Go RAM

Architecture AWS envisagée

- **S3** : stockage des données et artefacts
- **ECS Fargate** : API et ETL
- **RDS PostgreSQL** : données structurées
- **MongoDB Atlas** : métadonnées
- **MLflow** : suivi des expériences
- **ECR** : images Docker
- **CloudWatch** : logs et monitoring

Estimation budgétaire

Entre 350 et 700 €/mois

600 €/mois retenus

Enveloppe maximale : 1 000 €/mois

BC02 - Analyse exploratoire, descriptive et inférentielle de données

Présentation des datasets

GuitarSet [6]

► Dataset

- 360 extraits d'environ 30 s chacun.
- 6 guitaristes, 5 styles, 3 progressions, 2 tempi.
- Guitare acoustique et accordage standard.
- Captation hexaphonique et micro room.
- Audio WAV mono ou stéréo, 44100 Hz.
- Annotations temporelles au format JAMS.
- Licence Creative Commons Attribution 4.0 International.

IDMT-SMT-Guitar [4]

► Dataset

- 7 guitares en accordage standard.
- Différents micros et configurations.
- Audio WAV mono, 44100 Hz.
- Annotations au format XML.
- Techniques de jeu.
- Licence Creative Commons Attribution **Non Commercial No Derivatives** 4.0 International.

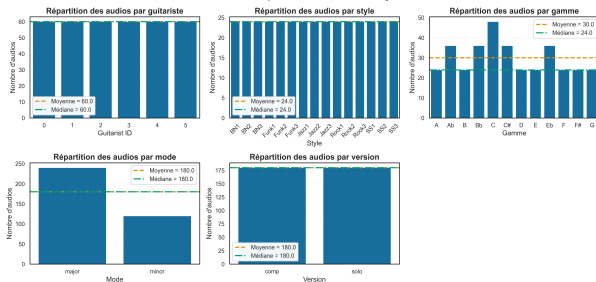
Conclusion

Les deux datasets sont complémentaires. GuitarSet contient des annotations et contextes musicaux riches et IDMT-SMT-Guitar apporte une diversité instrumentale et des techniques de jeu.

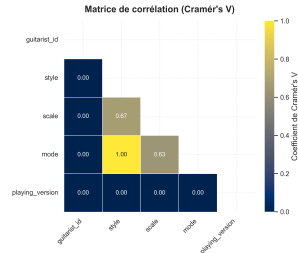
Analyse du dataset GuitarSet - Métadonnées

guitarist_id	title	style	tempo	scale	mode	playing_version	duration
0	00_BN1-129-Eb_comp	BN1	129	Eb	major	comp	22.32

Métadonnées - Analyse univariée - Variables catégorielles



Métadonnées - Analyse bivée
- Variables catégorielles

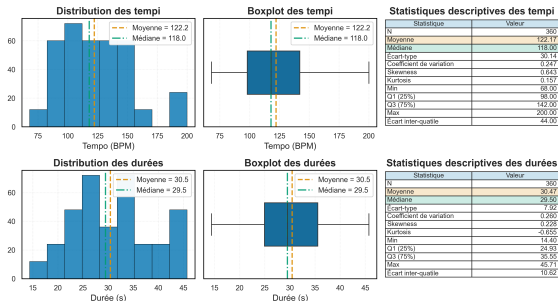


Bien que les styles musicaux soient équirépartis, le dataset présente un léger déséquilibre des classes sur les gammes et les modes.

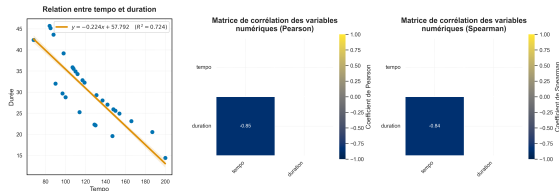
Chaque style est associé à un mode.
Chaque gamme semble préférentiellement associée à un style musical ou à un mode.

Analyse du dataset GuitarSet - Métadonnées

Métadonnées - Analyse univariée - Variables numériques



Métadonnées - Analyse bivariable - Variables numériques

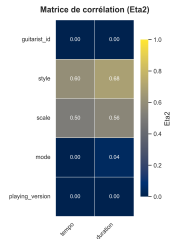


Il semble que plus le tempo d'un morceau est rapide, plus il est court et réciproquement. Cette relation est raisonnablement linéaire.

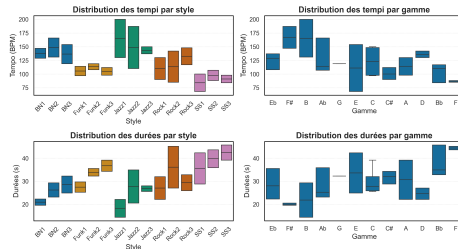
Les conditions sont favorables à une analyse de corrélation : absence d'outliers, distribution relativement symétrique, dispersions non pathologiques et kurtosis raisonnables.

Analyse du dataset GuitarSet - Métadonnées

Métadonnées - Analyse bivariable - Variables catégorielles et numériques



Métadonnées - Analyse bivariable - Variables catégorielles et numériques



Les styles musicaux semblent préférentiellement associés à des tempi et des durées.

Les styles présentent des profils rythmiques distincts et influencent la durée des extraits. Les étendues des tempi et des durées varient selon la gamme.

Conclusion

L'analyse des métadonnées montre des corrélations entre **style**, **gamme**, **tempo** et **durée**. Ces dépendances caractérisent la distribution du dataset et peuvent influencer l'apprentissage du modèle.

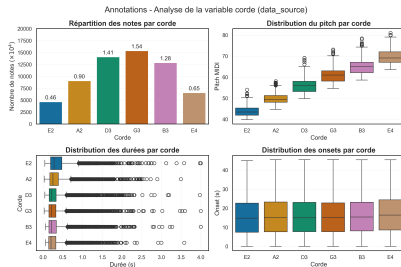
Analyse du dataset GuitarSet - Annotations

title	data_source	time	duration	value
00_BN1-129-Eb_comp	0	7.457274	0.464399	44.018917

Hauteur de notes par corde

string	min	theoretical	min	max	theoretical	max
E2	39.79	40	54.01	59	59	59
A2	44.68	45	58.06	64	64	64
D3	49.75	50	69.10	69	69	69
G3	54.63	55	73.09	74	74	74
B3	58.60	59	78.39	78	78	78
E4	63.58	64	80.98	83	83	83

Pas d'anomalie détectée : les intervalles des hauteurs de notes réelles et théoriques coïncident.



Les annotations semblent cohérentes avec la physique de l'instrument. La sous représentation des notes graves devrait être balancée par leurs durées. En revanche, la sous représentation des notes aiguës couplée à leur durée courte laisse présager des difficultés de transcription pour ces notes.

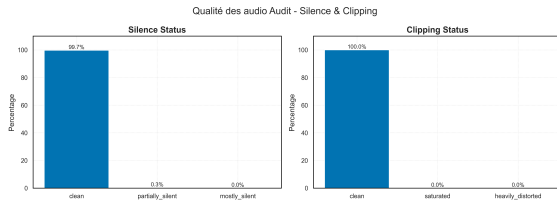
Analyse du dataset GuitarSet - Polyphonie et qualité audio

Activations simultanées

En considérant que des activations ayant au plus 20ms d'écart sont des notes activées simultanées, on peut estimer la polyphonie du dataset.

- En moyenne, 1,67 ($\pm 1,11$) notes activées simultanément.
- Au moins 25% d'intervalle ou d'accords.
- Au maximum, 6 notes actives simultanément, ce qui correspond à la physique de l'instrument.

Qualité Audio



Les audios sont propres, peu de silence et pas d'écroulement.

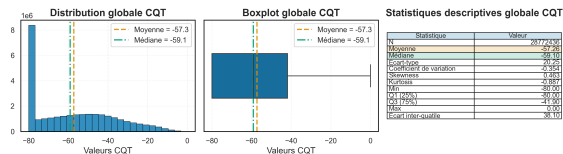
Conclusion

Dataset riche mais partiellement contraint. Bonne base pour apprentissage supervisé. IDMT-SMT-Guitar constitue un complément intéressant.

Analyse d'un dataset Frame-Wise - Features

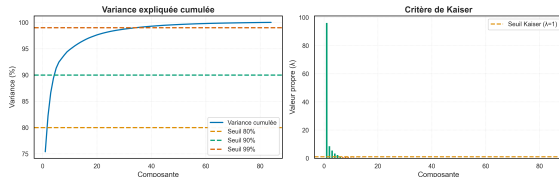
A partir des datasets GuitarSet et IDMT-SMT-Guitar, on construit un dataset frame-wise grâce à une transformée Q-Constante (CQT).

Features - Analyse univariée



Le dataset contient énormément de bins temps-fréquence sans énergie, cela implique un fort déséquilibre des données. Le modèle risque de ne pas détecter l'activation des notes. Il faudra privilégier une standardisation robuste : $x' = \frac{x - \text{médiane}}{IQR}$.

Features - Analyse multivariée - PCA

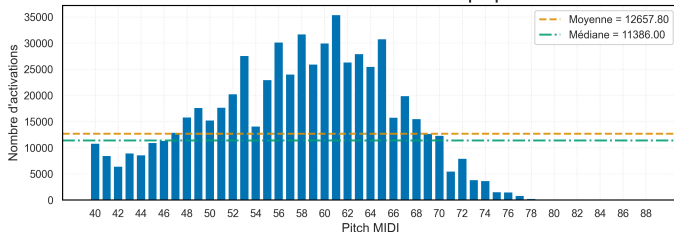


L'analyse des corrélations entre features permet d'envisager une PCA. 6 valeurs propres supérieures à 1. Elles expliquent au moins 92% de la variance.

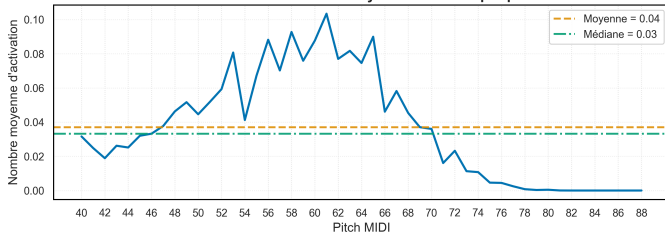
Analyse d'un dataset Frame-Wise - Target

Target - Analyse univariée

Distribution du nombre d'activations par pitch



Distribution du nombre moyen d'activation par pitch

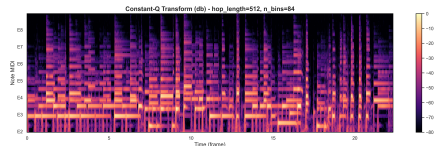
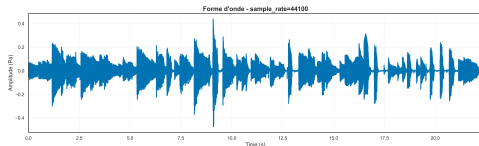


Le déséquilibre des classes est un réel problème, le modèle risque de ne pas détecter d'activation. La distribution des activations de notes est hétérogène, les notes aiguës seront plus difficiles à détecter.

BC03 - Analyse prédictive de données structurées par l'intelligence artificielle

Construction d'un dataset frame-wise avec CQT

Transformée Q constante



Dataset frame-wise

Features : CQT					Target : notes MIDI		
Frame	Bin 1	Bin 2	...	Bin N	E2	F2	...
t_1	0.12	0.03	...	0.00	0	1	...
t_2	0.15	0.04	...	0.00	0	1	...
$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$	$\vdots$	
t_T	0.00	0.00	...	0.00	0	0	...

Principe

Chaque ligne du dataset correspond à une frame temporelle. Les features CQT décrivent le contenu spectral du signal et la targer indique les notes actives.

Définition des modèles

- Plusieurs modèles de Machine Learning ont été implémentés selon une stratégie One-vs-Rest (OvR).
- Tous les modèles sont intégrés dans une pipeline comprenant : une normalisation **RobustScaler** et le classificateur OvR.
- Les classes sont pondérées afin de limiter l'impact du déséquilibre entre classes.
- Classificateurs entraînés : Logistic Regression, Linear SVM, Random Forest, HistGradientBoosting et SGD.

```
def get_ovr_hgb_model():  
    scaler = RobustScaler()  
    model = OneVsRestClassifier(  
        HistGradientBoostingClassifier(  
            learning_rate=0.1,  
            max_depth=8,  
            max_iter=200,  
            class_weight="balanced",  
            random_state=RANDOM_STATE,  
        )  
    )  
    return Pipeline(  
        steps=[  
            ("scaler", scaler),  
            ("model", model),  
        ]  
    )
```

Métriques et critères d'évaluation

Métriques ML

- **F1-micro** : qualité globale de la transcription
- **F1-macro** : généralisation sur l'ensemble du registre
- **Subset Accuracy** : frames parfaitement transcrites

seuil $\geq 90\%$

métrique de suivi

métrique de suivi

Métriques musicales

- **Hamming Loss** : erreur moyenne par note et par frame
- **Pitch Tolerance Accuracy** : proximité musicale
- **Jitter** : stabilité temporelle de la transcription

à minimiser

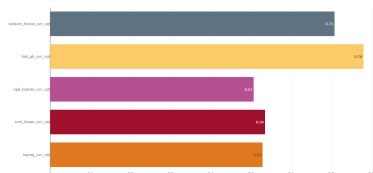
à maximiser

à minimiser

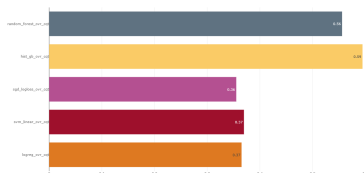
Critère de réussite du POC

F1-micro $\geq 90\%$

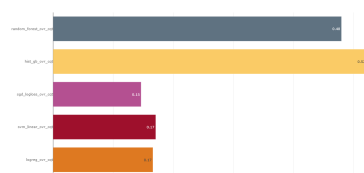
CV F1-micro



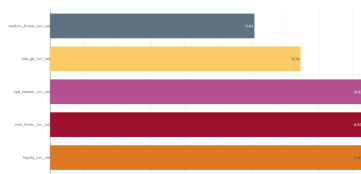
F1-macro



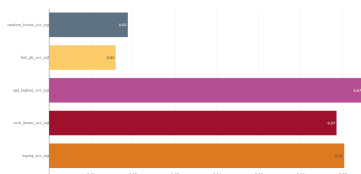
Subset accuracy



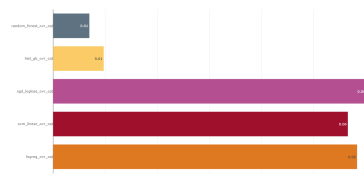
Pitch accuracy tolerance 1/2 ton



Hamming Loss



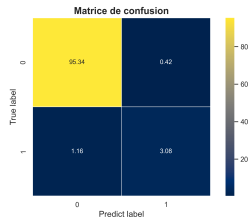
Jitter



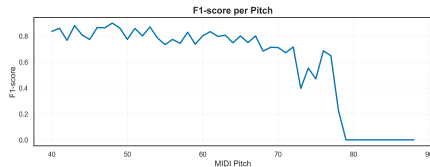
Conclusion

Le meilleur modèle est OvR + HistGradientBoosting.

Matrice de confusion



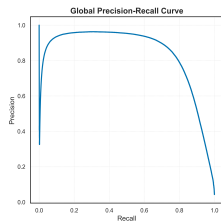
F1-micro par hauteur de note



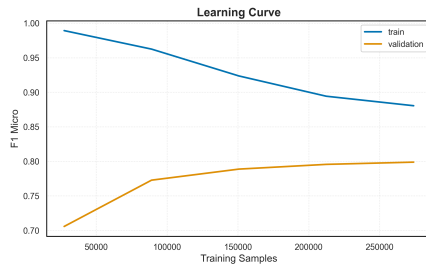
Conclusion

Globalement le modèle est correct pour une baseline mais insuffisant pour une transcription audio vers MIDI, il souffre d'un recall trop bas. La détection des notes aiguës est particulièrement difficile.

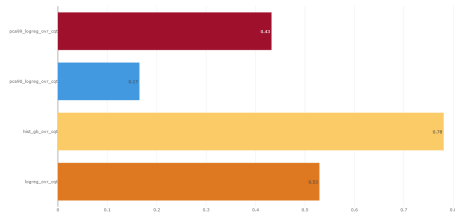
Courbe Precision / Recall



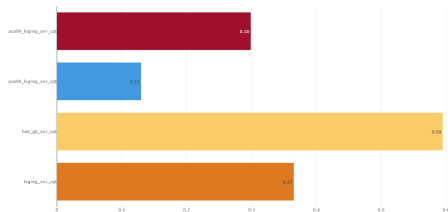
Courbe d'apprentissage



CV F1-micro



F1-macro

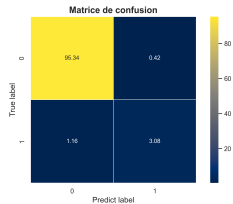


Conclusion

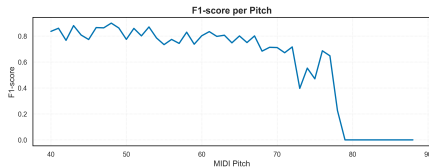
L'ajout d'une PCA dégrade les performance du modèle OvR + LogisticRegression. La structure des données semble essentielle à l'apprentissage du modèle. Les combinaisons linéaires des features dégradent l'apprentissage.

Optimisation - RandomizedSearchCV

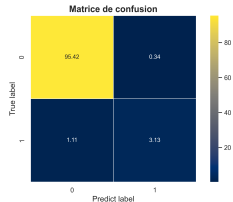
Baseline



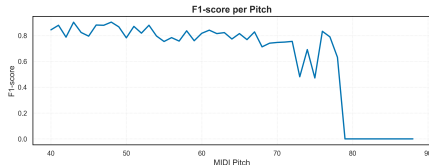
Baseline



Modèle optimisé



Modèle optimisé



Conclusion

L'optimisation est réussie, le modèle optimisé affiche de meilleures performances dans toutes les métriques. Cependant, il ne parvient pas à corriger suffisamment le recall du modèle initial, on se tournera donc vers des modèles plus complexes type MLP pour la suite des expérimentations.

BC04 - Analyse prédictive de données non-structurées par l'intelligence artificielle

Métriques et critères d'évaluation

Métriques ML

- **F1-micro** : qualité globale de la transcription
- **F1-macro** : généralisation sur l'ensemble du registre
- **Subset Accuracy** : frames parfaitement transcrites

seuil $\geq 90\%$

métrique de suivi

métrique de suivi

Métriques musicales

- **Hamming Loss** : erreur moyenne par note et par frame
- **Pitch Tolerance Accuracy** : proximité musicale
- **Jitter** : stabilité temporelle de la transcription

à minimiser

à maximiser

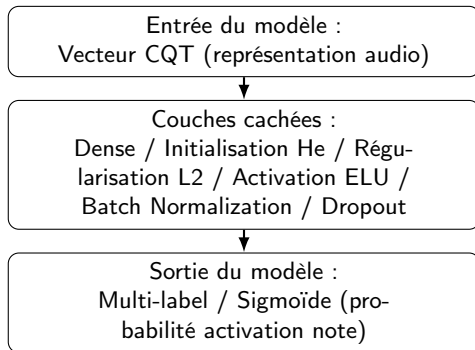
à minimiser

Critère de réussite du POC

F1-micro $\geq 90\%$

Architecture du modèle MLP

Architecture



Apprentissage

- **Fonction de coût** : Binary Cross-Entropy
- **Évaluation** : Precision, Recall, F1-score micro

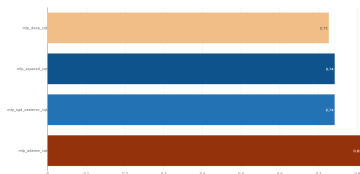
Variantes étudiées

- **AdamW** : Architecture de référence
- **SGD + Nesterov** : Optimiseur alternatif
- **Squared** : Couches cachées de même largeur
- **Deep** : Augmentation de la profondeur

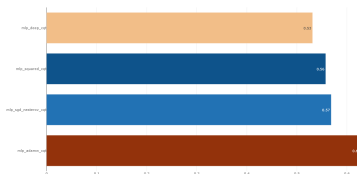
Objectif expérimental

Étudier l'influence de l'architecture et de l'algorithme d'optimisation sur la transcription polyphonique.

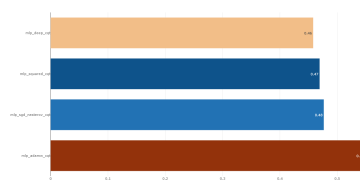
F1-micro



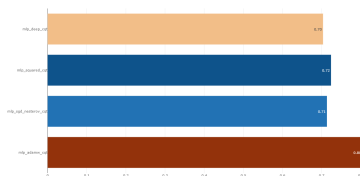
F1-macro



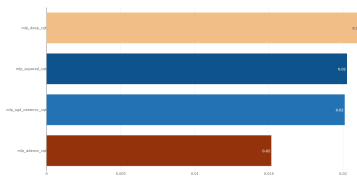
Subset accuracy



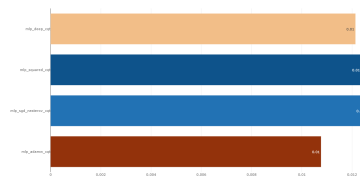
Pitch accuracy tolerance 1/2 ton



Hamming Loss



Jitter

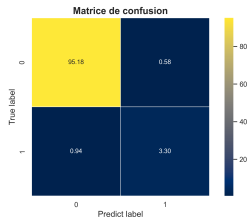


Conclusion

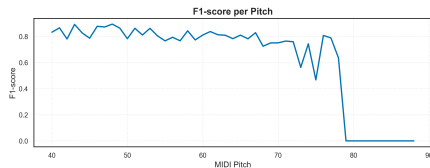
Le modèle MLP avec optimiseur AdamW montre les meilleures performances.

Analyse du modèle MLP avec optimiseur AdamW

Matrice de confusion



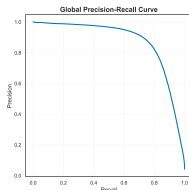
F1-micro par hauteur de note



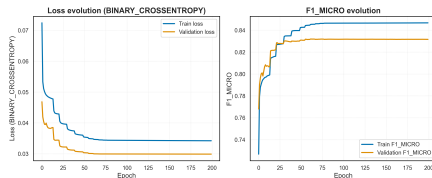
Conclusion

Le modèle MLP avec optimiseur AdamW montre de meilleures performances que le modèle OvR + HistGradientBoosting. Il gagne en Recall mais perd en Precision tout en augmentant le F1-score.

Courbe Precision / Recall



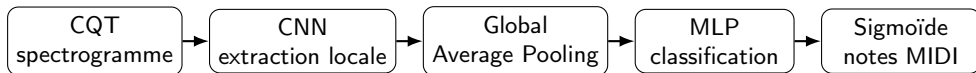
Historique d'apprentissage



Pour améliorer les performance du modèle, on propose d'ajouter une fenêtre de contexte et des couches de convolutions (CNN).

Architecture CNN + MLP

Architecture



Apport par rapport au MLP

- Exploitation de la structure temps–fréquence de la CQT.
- Convolutions à différentes échelles.
- Pooling pour réduire la dimension.
- Global Average Pooling avant le MLP.

Variantes expérimentées

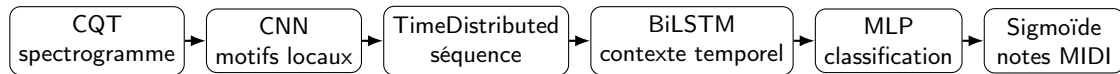
- **Light** : coût de calcul réduit
- **Balanced** : compromis capacité / coût
- **Frequency** : contexte harmonique élargi

Objectif expérimental

Étudier l'influence de l'exploitation des motifs locaux temps-fréquence.

Architecture RCNN : CNN + BiLSTM + MLP

Architecture



Apport par rapport au CNN + MLP

- Conservation de la dimension temporelle.
- Transformation des cartes CNN en séquence.
- BiLSTM : exploitation du contexte passé et futur.
- Meilleure modélisation des dépendances temporelles entre notes.

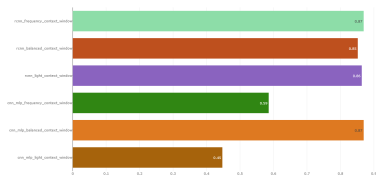
Variantes expérimentées

- **Light** : coût de calcul réduit
- **Balanced** : compromis capacité / coût
- **Frequency** : contexte harmonique élargi

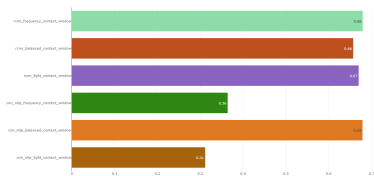
Objectif expérimental

Etudier l'influence de l'ajoute de la dynamique temporelle.

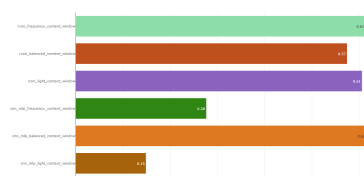
F1-micro



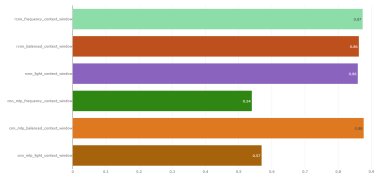
F1-macro



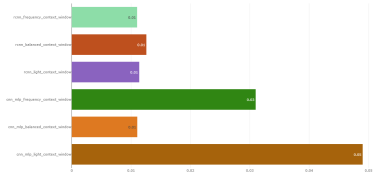
Subset accuracy



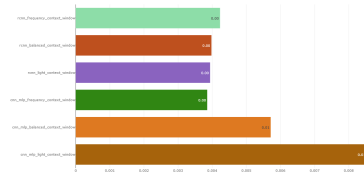
Pitch accuracy tolerance 1/2 ton



Hamming Loss



Jitter

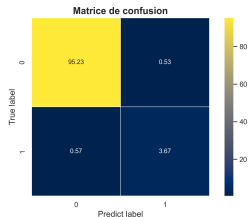


Conclusion

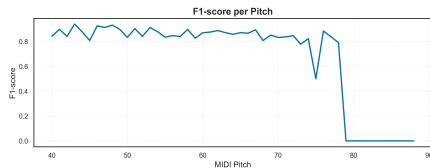
Deux modèles se distinguent par leurs performances : CNN + MLP balanced et RCNN frequency.

Analyse du modèle RCNN frequency

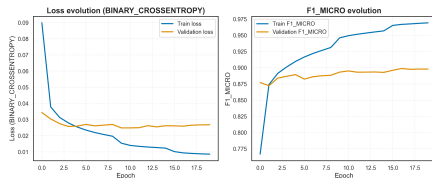
Matrice de confusion



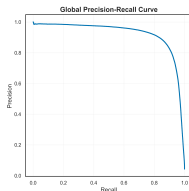
F1-micro par hauteur de note



Historique d'apprentissage



Courbe Precision / Recall



Conclusion

L'ajout d'une fenêtre de contexte a permis d'améliorer le recall sans sacrifier la précision.

Le modèle RCNN frequency est retenu pour la suite du projet : sa capacité à modéliser les dépendances temporelles laisse envisager un potentiel de généralisation supérieur au modèles CNN + MLP, qui pourrait s'exprimer avec un volume de données plus important.

BC05 - Industrialisation d'un algorithme d'apprentissage automatique et automatisa- tion des processus de décision

Architecture de déploiement



Pipeline de déploiement

- Sélection du meilleur modèle et intégration à l'API.
- CI valide la qualité du projet.
- CD construit l'image Docker et la publie sur GHCR.
- Mise à jour du Space Hugging Face.

Conclusion

Environnement d'exécution identique entre développement, CI/CD et production.

Standardisation de l'environnement

Environnement reproductible

- Docker
- pyproject.toml
- uv.lock
- dépendances versionnées

Optimisation

- Dockerfile multi-stage
- utilisateur non privilégié

Résultat

L'utilisation conjointe de Docker et uv garantit un environnement standardisé, reproductible et portable.

```
FROM ghcr.io/astral-sh/uv:python3.13-bookworm-slim AS builder
WORKDIR /app
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy UV_NO_CACHE=1
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev --group api
COPY api/backend ./backend
COPY api/frontend ./frontend
COPY api/.streamlit ./streamlit
COPY api/start.sh ./start.sh

FROM python:3.13-slim-bookworm AS runtime
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends libsndfile1 libcairo2 && rm -rf /var/lib/apt/lists/*
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 PATH="/app/.venv/bin:${PATH}"
COPY --from=builder /app/.venv ./venv
COPY --from=builder /app/backend ./backend
COPY --from=builder /app/frontend ./frontend
COPY --from=builder /app/.streamlit ./streamlit
COPY --from=builder /app/start.sh ./start.sh
RUN chmod +x start.sh
RUN mkdir -p /tmp
RUN useradd --create-home --shell /usr/sbin/nologin appuser && chown -R appuser:appuser /app && chmod 777 /tmp
USER appuser
EXPOSE 7860
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health')" || exit 1
CMD ["/start.sh"]
```

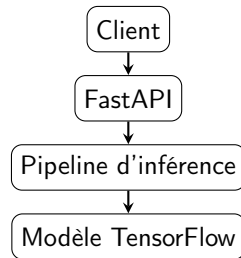
Exposition du modèle par une API REST

API REST

- Développée avec FastAPI.
- Architecture logicielle reposant sur une séparation des responsabilités.
- Modèle chargé au lancement de l'API et partagé entre les requêtes.
- Réponse normalisées au travers d'un modèle générique.
- Exceptions centralisées.
- Documentation automatiquement avec OpenAPI.

Endpoints principaux

- /health
- /model
- /predict
- /artifact/{id}/...



Objectif

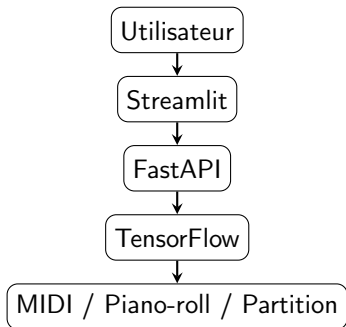
Découpler le modèle de l'interface afin de rendre les prédictions accessibles à d'autres applications.

Parcours utilisateur

- Déposer un fichier WAV
- Lancer la transcription via requête API
- Visualiser le résultat
- Télécharger les artefacts

Artefacts téléchargeables

- fichier MIDI,
- piano-roll,
- partition musicale.



Résultat

L'interface Streamlit se limite à la collecte des données utilisateur, à l'appel des services API REST et à la restitution des résultats.

Démonstration

► [Hugging Face Spaces](#)

BC06 - Direction de projets de gestion de données

Présentation du projet

GuitarFlow est une entreprise développant un prototype de service d'intelligence artificielle destiné à la transcription automatique d'enregistrements audio de guitare en fichiers MIDI exploitables en MAO (Musique Assistée par Ordinateur).

Constat métier

La retranscription manuelle d'une performance de guitare vers un format éditable constitue une tâche **chronophage**, nécessitant une **expertise musicale avancée** et difficilement **industrialisation-compatible** pour des usages pédagogiques ou créatifs.

Opportunité

Le projet vise à :

- réduire le temps de transcription audio → MIDI,
- faciliter la production de contenus pédagogiques,
- accélérer les workflows de composition musicale,
- proposer un démonstrateur d'IA appliquée au signal audio.

Objectif principal

Développer un prototype de service capable de :

- recevoir un fichier audio guitare,
- générer automatiquement un fichier MIDI exploitable,
- produire une visualisation de type piano-roll,
- exposer le modèle via une API web déployée sur Hugging Face.

Fonctionnalités incluses

- Audio guitare mono instrument
- Guitare acoustique et électrique
- Accordage standard (EADGBE)
- Génération MIDI
- Visualisation piano-roll
- API REST d'inférence
- Conteneurisation Docker

Fonctionnalités hors périmètre

- Multi-instruments
- Accordages alternatifs
- Temps réel
- Techniques de jeu avancées
- Application mobile native
- Architecture cloud managée

Analyse des besoins et exigences

Exigences fonctionnelles

- Accepter un enregistrement audio de guitare au format WAV.
- Produire automatiquement une transcription au format MIDI exploitable dans un logiciel de MAO.
- Permettre la visualisation des notes détectées sous forme de piano-roll.
- Exposer le service via une API REST et une interface web.
- Garantir la prise en charge d'enregistrements de guitare acoustique et électrique.
- Permettre l'évaluation et la comparaison de plusieurs approches de transcription.

Exigences non fonctionnelles

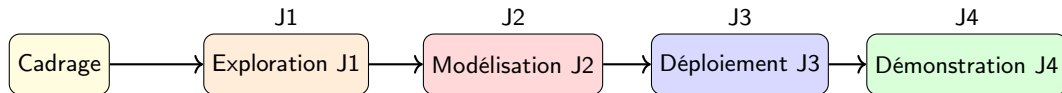
- Temps d'inférence inférieur à 10 secondes pour 1 minute d'audio.
- Reproductibilité complète des expérimentations et des résultats.
- Traçabilité des jeux de données, modèles et métriques.
- Déploiement reproductible par conteneurisation Docker.
- Maintenabilité assurée par une architecture modulaire, des tests automatisés et une documentation technique.

Indicateur	Objectif cible
F1-score de transcription polyphonique	$\geq 90 \%$
Temps moyen d'inférence	< 10 s pour 1 min d'audio
Disponibilité du prototype	Démonstration fonctionnelle avant le 31/07/2026
Déploiement	API accessible via une interface web
Taux de correction manuelle	$< 10 \%$

Approche de pilotage

Le projet a été conduit selon une démarche incrémentale articulée autour de quatre jalons de validation permettant de sécuriser progressivement les différentes phases du POC.

Jalon	Semaine	Critère de validation
J1 – Go Datasets	S4	Données validées et exploitables
J2 – Go Modèle	S6	Baseline opérationnelle
J3 – Go Prototype	S8	Pipeline complet exécutable
J4 – Go Démonstration	S10	API et interface accessibles



► Voir le Gantt détaillé

Risque	Impact	Prob.	Mitigation
Restrictions de licence sur les data-sets	Élevé	Moyen	Validation juridique des sources avant intégration
Volume de données insuffisant	Élevé	Élevé	Data augmentation + transfer learning
Performances insuffisantes en polyphonie	Élevé	Moyen	Benchmark de plusieurs architectures (baseline / CNN / CRNN)
Temps d'inférence trop élevé	Moyen	Moyen	Optimisation du modèle + quantization
Variabilité de qualité audio	Moyen	Élevé	Normalisation et standardisation du signal audio

Contraintes budgétaires et choix d'architecture

Contrainte principale

Le projet s'inscrit dans une logique de prototypage à faible coût, orientée preuve de concept.

Hypothèse budgétaire : valorisation de la charge projet à 500 € / jour, soit 25 000 € pour 50 jours.

Principes d'architecture retenus

- Priorité aux solutions open source
- Exécution locale ou infrastructure légère
- Limitation des coûts de calcul et d'entraînement
- Absence d'infrastructure cloud industrielle en phase 1

Conséquences techniques

- Utilisation de Docker Compose pour orchestration locale
- Stockage local / MinIO plutôt que cloud managé
- Choix de modèles compatibles CPU

Contraintes réglementaires et licences

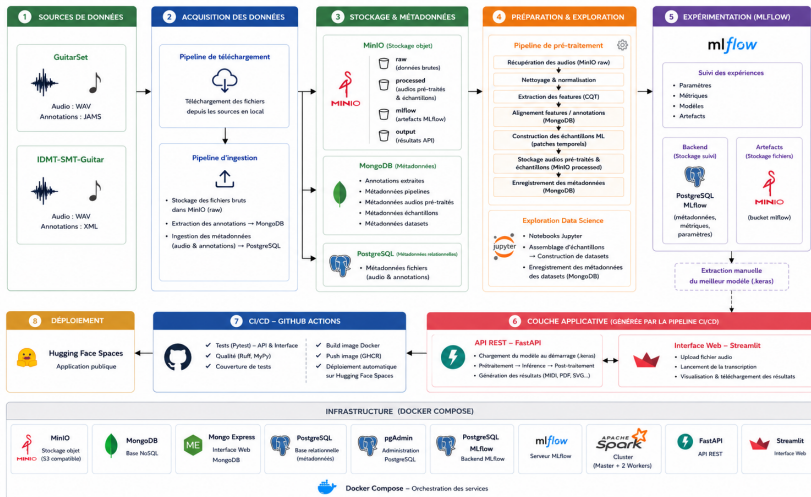
- GuitarSet : licence compatible Creative Commons Attribution 4.0 International
- IDMT-SMT-Guitar : licence non commerciale limitant l'usage à un POC

Impact projet

Le POC privilégie la reproductibilité et la maîtrise des coûts plutôt que la scalabilité industrielle.

ARCHITECTURE GLOBALE DE LA PLATEFORME

Guitarflow – Projet de transcription musicale



Réalisation du POC

Approche de réalisation

Le POC a été construit selon une logique de pipeline end-to-end couvrant l'ensemble du cycle de vie de la donnée : ingestion, traitement, entraînement et mise à disposition du modèle.

Livrables techniques

- Pipelines de téléchargement et d'ingestion des données
- Pipeline de prétraitement audio et extraction de caractéristiques
- Pipeline de construction du dataset frame-wise
- Pipeline d'entraînement et d'évaluation des modèles
- Modèle entraîné et évalué pour la transcription audio → MIDI
- API REST d'inférence (FastAPI)
- Conteneurisation du système via Docker / Docker Compose
- Interface de démonstration déployée sur Hugging Face Spaces

Livrables documentaires

- Note de cadrage et cahier des charges
- Documentation technique des pipelines et de l'architecture
- Guide d'installation et de déploiement

Performance du système

Indicateur	Résultat
F1-score transcription polyphonique	87%
Temps d'inférence moyen	93s pour 1 min d'audio
Génération de fichier MIDI	Oui
Visualisation piano-roll	Oui
API d'inférence fonctionnelle	Oui
Interface de démonstration déployée	Oui
Taux de correction manuelle	39 %

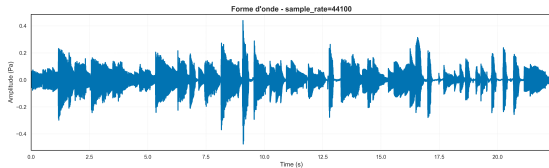
Synthèse

Bien que les résultats sont en deçà des espérances, le POC démontre globalement la faisabilité technique de la transcription audio → MIDI.

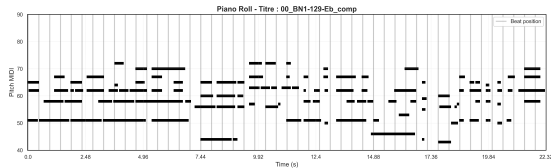
Exemple de transcription audio vers MIDI

Transformation réalisée par le système

Audio (.wav) → Modèle de transcription → MIDI (.mid)



Forme d'onde de l'enregistrement guitare



Piano-roll généré à partir de la transcription MIDI

Synthèse

Le système transforme automatiquement un signal audio brut en représentation symbolique exploitable par les logiciels de MAO.

Gouvernance et qualité

Gouvernance des données

- Datasets stockés en brut dans MinIO.
- PostgreSQL utilisé comme source de vérité.

Traçabilité des traitements

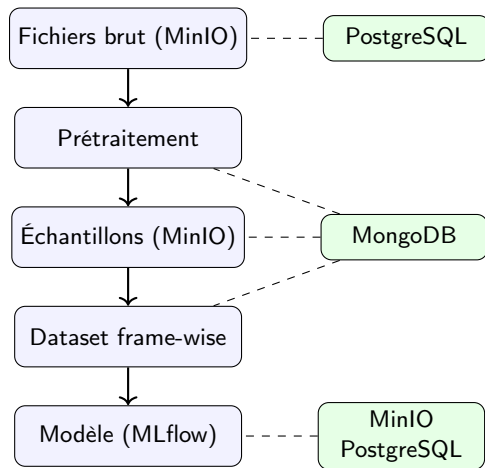
- Métadonnées stockées dans MongoDB.
- Versionnement des prétraitements et des datasets.
- Tracking des expérimentations.

Qualité logicielle

- GitHub / Docker / uv / Ruff + Mypy / Tests

Résultat

Chaque modèle peut être relié aux données sources, aux prétraitements appliqués et aux dataset frame-wise sur lequel il s'est entraîné.



Traçabilité complète des données, traitements et modèles.

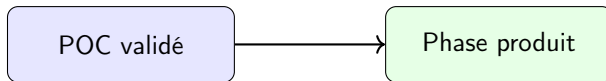
Conclusion

Bilan du projet

- Faisabilité technique démontrée
- Pipeline end-to-end opérationnel
- Génération automatique de fichiers MIDI
- API REST et interface web déployées
- Traçabilité et reproductibilité assurées

Perspectives

- Amélioration des performances en polyphonie complexe
- Optimisation des temps d'inférence
- Déploiement d'une API dédiée en production
- Détection avancée des techniques de jeu
- Support multi-instruments



Conclusion

Le projet valide la faisabilité d'une solution de transcription audio vers MIDI et constitue une base solide pour une phase d'industrialisation.

Je vous remercie pour votre attention.
Avez-vous des questions ?

9. Diagramme de Gantt détaillé

10. Stack technologique du projet

11. Bibliographie

Diagramme de Gantt détaillé

Phase	Tâches	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
Cadrage	Validation note de cadrage										
Exploration	Ingestion des données										
	Analyse exploratoire des données										
	Preprocessing audio et extraction des features										
	Construction des datasets frame-wise				J1						
Modélisation	Benchmark des modèles candidats						J2				
	Implémentation du pipeline ML complet								J3		
Déploiement	Développement de l'API d'inférence										
	Dockerisation et déploiement sur Hugging Face Spaces										
Démonstration	Présentation phase1										J4

Stack technologique du projet

Groupe	Thème	Technologie (version)
Data Engineering	Stockage objet Base relationnelle Base document	MinIO (latest) PostgreSQL (latest) MongoDB (latest)
Traitement des données	Calcul distribué Data manipulation Calcul scientifique Audio processing Annotation audio MIDI processing	Apache Spark (latest) Pandas (latest) NumPy (latest) Librosa (latest) JAMS (latest) pretty_midi (latest)
Machine Learning	Machine Learning Deep Learning Tracking ML	Scikit-Learn (latest) TensorFlow (latest) MLflow (latest)
API & Déploiement	API REST Schéma data Conteneurisation Interface web	FastAPI (latest) Pydantic (latest) Docker (latest) Streamlit (latest)



Aurélien Géron.

Machine Learning avec Scikit-Learn : mise en oeuvre et cas concrets.

Dunod, Paris, 3 edition, 2023.



Aurélien Géron.

Deep Learning avec Keras et TensorFlow : mise en oeuvre et cas concrets.

Dunod, Paris, 3 edition, 2024.



Christian Kehling, Jakob Abeßer, Christian Dittmar, and Gerald Schuller.

Automatic tablature transcription of electric guitar recordings by estimation of score- and instrument-related parameters.

In *Proceedings of the 17th International Conference on Digital Audio Effects (DAFx)*, Erlangen, Germany, 2014.



Christian Kehling, Andreas Männchen, and Arndt Eppler.

Idmt-smt-guitar dataset, 2023.



Gilbert Saporta.

Probabilités, analyse des données et statistique.

Technip, Paris, 3 edition, 2011.



Qiaoxi Xi, Seth Dittmer, Justin Pauwels, and Mark Sandler.

Guitarset : A dataset for guitar transcription.

In *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, Paris, France, 2018.